

Tecnofor

IFCD0062

Fundamentos de Programación en Python

Formador: Ana Isabel Vegas

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL
fundae.es
SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE

Tecnofor

Temario:

1. Introducción a Python y a la programación
2. Elementos y operaciones de programación
3. Funciones, tuplas, diccionarios y procesamiento de datos
4. Módulos, paquetes, métodos y excepciones
5. Introducción a la POO: clases, objetos, excepciones y archivos

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL
fundae.es
SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE

Tecnofor

Modulo 1:

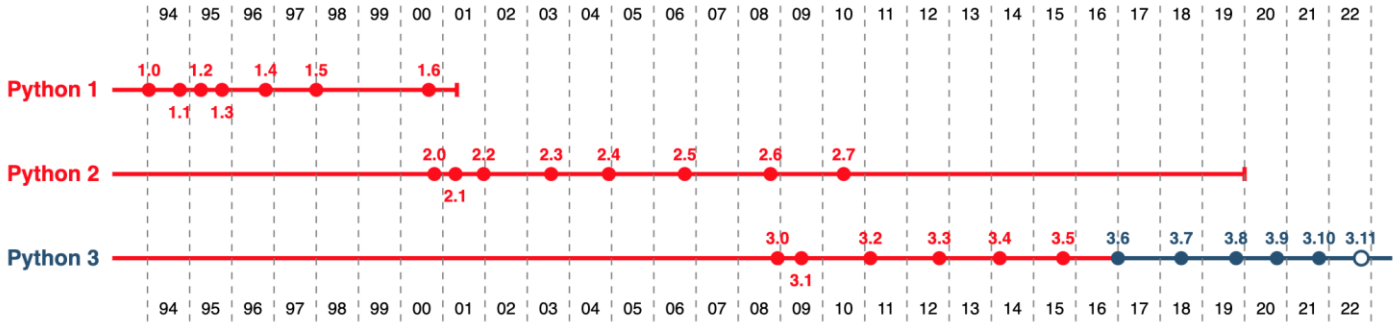
Introducción a Python y a la programación

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL
fundae.es
SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE

Qué es Python?

- Python es un lenguaje de programación que fue creado a finales de los años 80 por el holandés Guido van Rossum, fan del grupo humorístico Monty Python, de ahí el nombre que le puso al lenguaje de programación.

Versiones de Python



Características de Python

- **Simpleza**: es un lenguaje muy sencillo, de ahí el gran exito de Python.
- **Sintaxis clara**: es obligatoria la indentación
- **Propósito general**: se pueden crear todo tipo de programas
- **Lenguaje interpretado**: no es necesaria compilación, el interprete de Python lo ejecuta.
- **Lenguaje de alto nivel**: no hay que preocuparse del manejo de memoria y otros aspectos de bajo nivel
- **Lenguaje orientado a objetos**: Favorece la reutilización de código
- **Open source**: es un lenguaje de programación gratuito
- **Extensas librerías**: incluyen muchas funcionalidades
- **Incrustable**: es posible añadir programas Python a programas en C y C++.

Ventajas

- **Legible**: sintaxis intuitiva y estricta
- **Productivo**: ahorra mucho código
- **Portable**: para todo sistema operativo
- **Recargado**: viene con muchas librerías por defecto

Instalación en Windows

- Desde la página www.python.org/download se descarga y se instala.
- El sitio reconocerá el sistema operativo y te dará las opciones para descargar.



Comprobar versión instalada

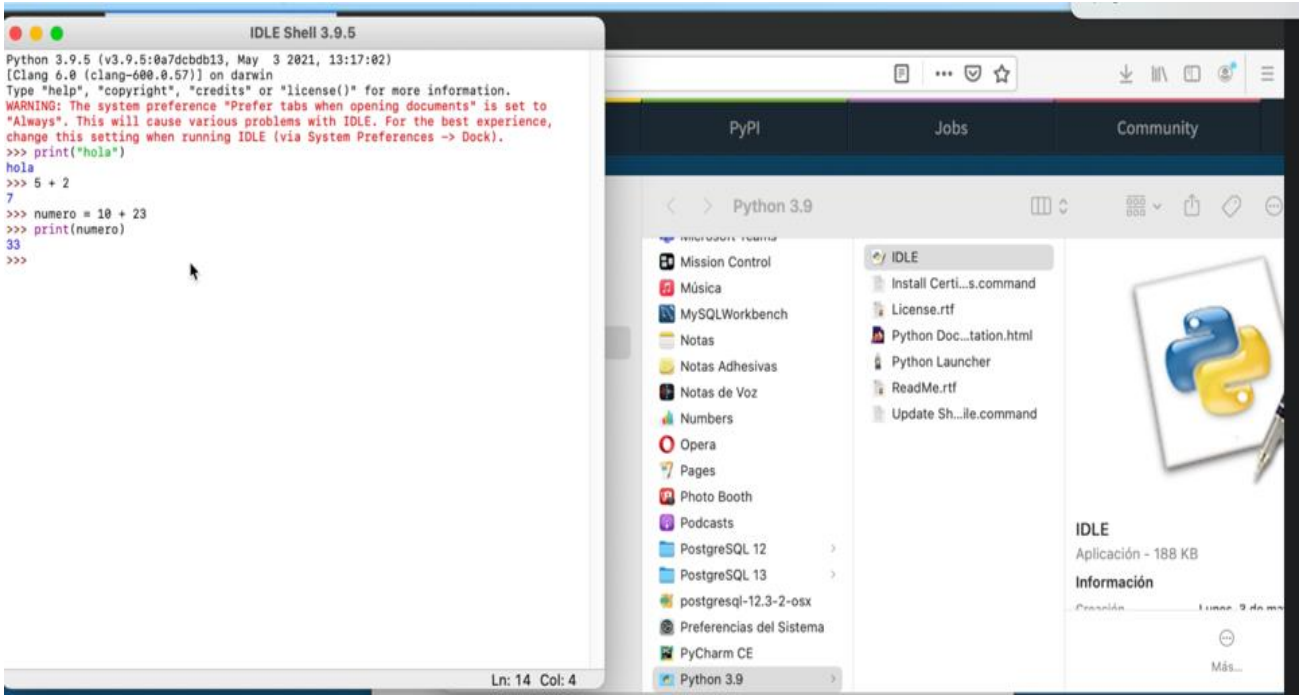
- Desde la terminal de comandos:

```
anabel@MacBook-Air-de-Anabel ~ % python3 --version
Python 3.14.3
```

Principales entornos de desarrollo

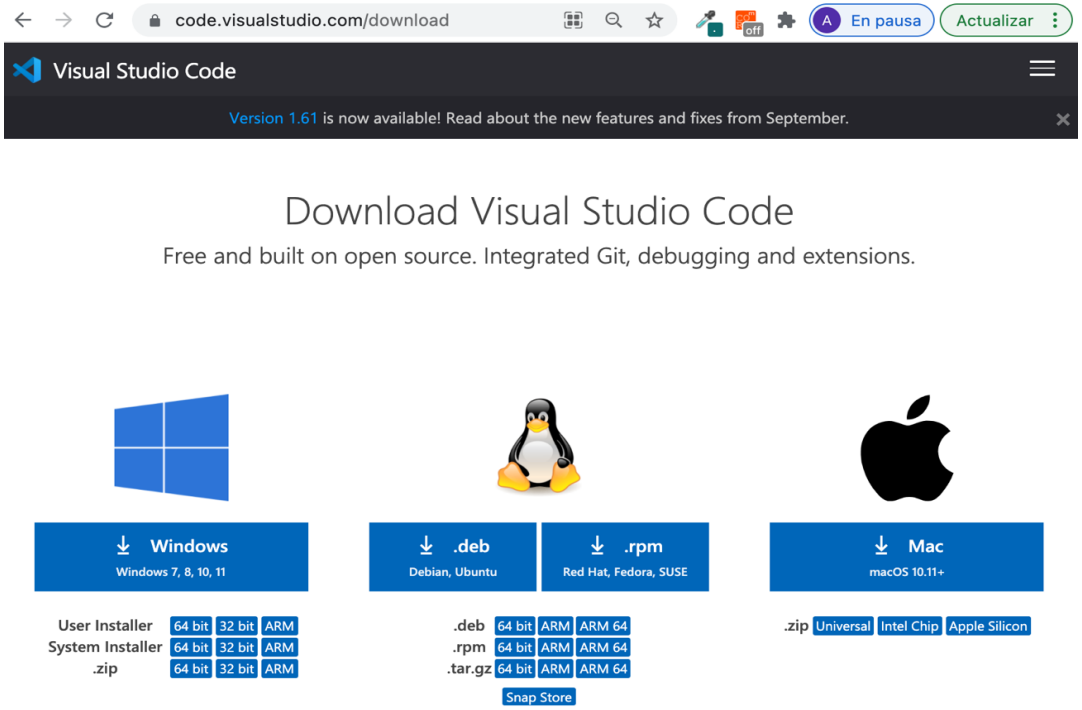
- Son muchos los entornos que podemos utilizar:
  - IDLE
  - Visual Studio Code
  - Jupiter
  - Anaconda
  - Spyder
  - PyCharm

IDLE

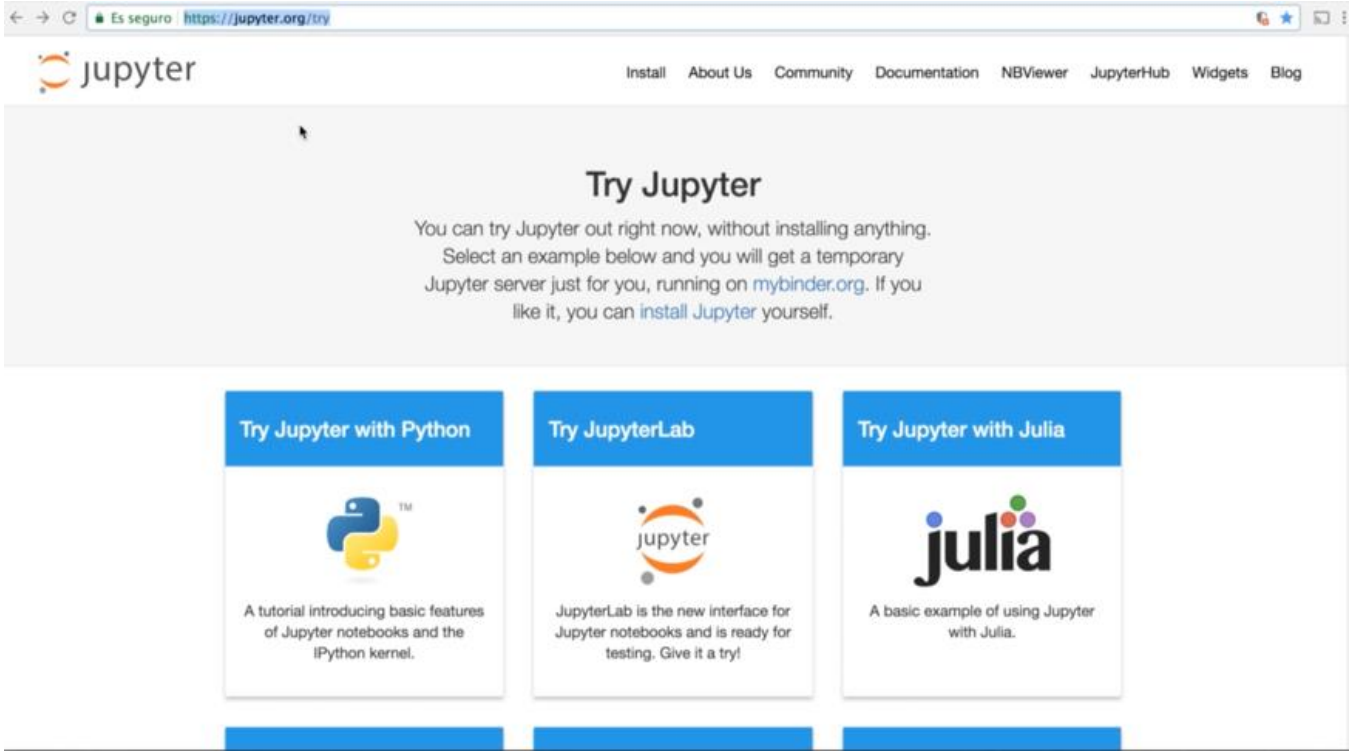


Para salir cerrar ventana o Ctrl + D.

Visual Studio Code



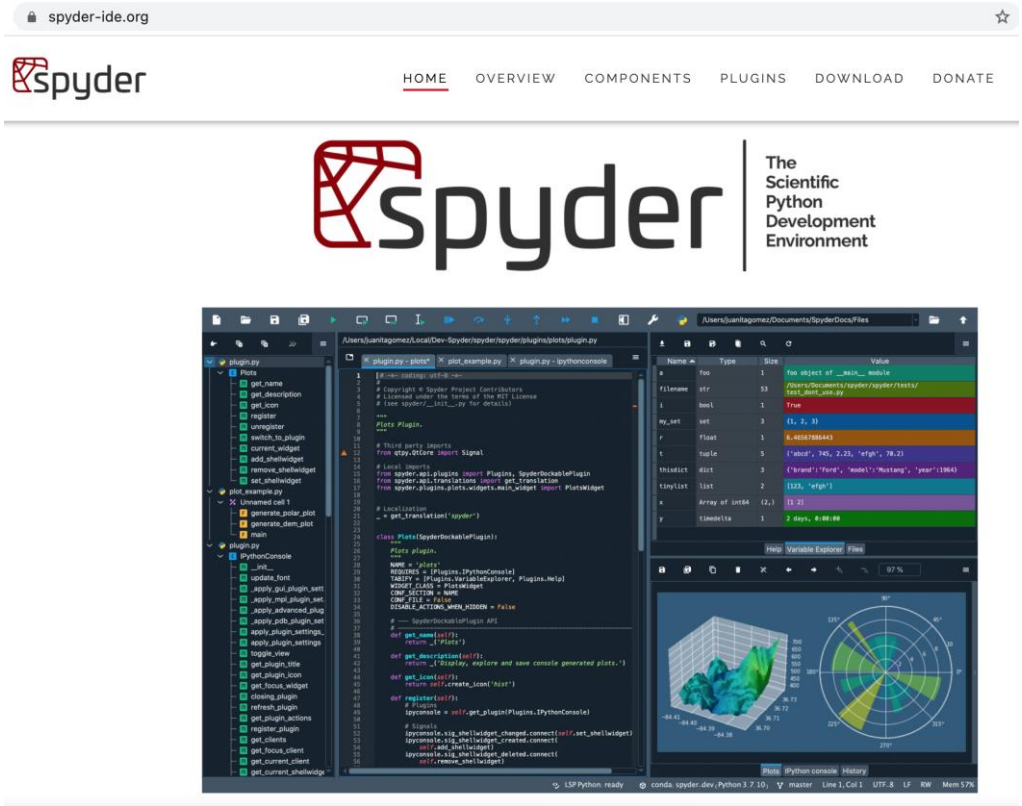
Jupiter


Es seguro https://jupyter.org/try
jupyter
Install About Us Community Documentation NBViewer JupyterHub Widgets Blog
Try Jupyter
You can try Jupyter out right now, without installing anything.
Select an example below and you will get a temporary
Jupyter server just for you, running on mybinder.org. If you
like it, you can install Jupyter yourself.
Try Jupyter with Python
A tutorial introducing basic features of Jupyter notebooks and the IPython kernel.
Try JupyterLab
JupyterLab is the new interface for Jupyter notebooks and is ready for testing. Give it a try!
Try Jupyter with Julia
A basic example of using Jupyter with Julia.

Anaconda


QuickTime Player Archivo Edición Visualización Ventana Ayuda
Home - Anaconda
https://www.anaconda.com
ANACONDA
Documentation Blog Contact
What is Anaconda? Products Support Resources About
Downloads
Report: 2018 Anaconda State of Data Science
Download Report
The Most Popular Python Data Science Platform
Accelerate
Streamline your data science workflows from data ingest through deployment
Connect
Leverage & integrate all your data sources to extract the most value from your data
Empower
Create, collaborate & share with your entire team—from analysts to executives

Spyder


spyder-ide.org
spyder
HOME
OVERVIEW
COMPONENTS
PLUGINS
DOWNLOAD
DONATE
spyder
The Scientific Python Development Environment

PyCharm


jetbrains.com/es-es/pycharm/
JET BRAINS
Para desarrolladores
Para equipos
Para aprender
Soluciones
Tienda
PyCharm
Próximamente en 2021.3
Novedades
Funcionalidades
Aprender
PC
PyCharm
IDE de Python
para desarrolladores
profesionales
DESCARGAR
Versión Professional completa o versión gratis Community

Declaración de variables

- Las variables, a diferencia de los demás lenguajes de programación, no debes definirlas, ni tampoco su tipo de dato, ya que al momento de iterarlas se identificará su tipo.
- Debe comenzar por letra o _
- Es case sensitive, diferencia mayúsculas de minúsculas
- Para saber el tipo de dato type(variable)

```
A = 3
B = A
```

```
numero1 = 5
numero2 = 4
suma = numero1 + numero2
```

Tipos de datos

Tipo de dato	Descripción
Entero	Número sin decimales, tanto positivo como negativo, incluyendo el 0.
Real	Número con decimales, tanto positivo como negativo, incluyendo el 0.
Complejo	Número con parte imaginaria.
Cadenas de texto	Texto.
Booleanos	Pueden tener dos valores: True o False
Conjuntos	Colección de elementos no ordenados y no repetidos.
Lista	Vector de elementos que pueden ser de diferentes tipos de datos.
Tuplas	Lista inmutable de elementos.
Diccionario	Lista de elementos que contienen claves y valores.

Conversiones entre tipos de datos

- **Números enteros**

- Toda entrada de usuario se interpreta como una cadena de texto, para poder transformarlo en un numero, se utiliza la instrucción int.

```
sumando1 = int(input("Introduzca el primer sumando: "))
sumando2 = int(input("Introduzca el segundo sumando: "))
print("Resultado de la suma: ", sumando1 + sumando2)
```

Conversiones entre tipos de datos

- **Números reales**
- Para transformar una cadena de texto en un número real, utilizamos la instrucción float.

```
sumando1 = float(input("Introduzca el primer sumando (Decimal): "))
sumando2 = float(input("Introduzca el segundo sumando (Decimal): "))
print("Resultado de la suma: ", sumando1 + sumando2)
```

- Para redondear números reales utilizamos la instrucción round que utiliza dos parámetros:
  - El primero de ellos es el número real que quieres redondear
  - El segundo es el número de decimales al que quieres redondear.

```
dividendo = float(input("Introduzca el dividendo: "))
divisor = float(input("Introduzca el divisor: "))
resultado = round(dividendo / divisor,1)
print("Resultado de la division: ", resultado)
```

Conversiones entre tipos de datos

- **Cadenas de texto**

- Para convertir un numero a cadena de texto y asi mostrarlo por pantalla utilizamos el comando str:

```
print("Has escrito el numero " + str(numero))
```

Cadenas de texto

- Las cadenas de texto son arrays de caracteres.
- Deben ir entre comillas

- Además se pueden utilizar los siguientes caracteres especiales:
  - \n: implica un salto de línea dentro de la cadena de texto.
  - \\: carácter para introducir la barra \ en la cadena de texto.
  - \': carácter para introducir una comilla simple en la cadena de texto.
  - \": carácter para introducir una comilla doble en la cadena de texto.
  - \t: carácter para introducir una tabulación horizontal en la cadena de texto.

Imprimir cadenas de texto

- El comando print lo utilizamos para mostrar información por pantalla. Podemos mostrar uno o varios elementos separados por comas.
- A través de este comando podemos utilizar 2 operadores:
  - sep; Nos permite establecer una cadena de texto que actuará como separador de los elementos a mostrar.
  - end; Será la cadena que se muestre al final.

```
# Comando print

print("Hola")
print("Juan", "Pedro", "Maria", "Luis")

#Parametro sep
print("Juan", "Pedro", "Maria", "Luis", sep=' | ')

#Parametro end
print("Juan", "Pedro", "Maria", "Luis", sep=',', end='.')
```

```
Python 3.7.7 (v3.7.7:d7c567b08f
[Clang 6.0 (clang-600.0.57)] on
Type "help", "copyright", "cred
>>>
= RESTART: /Users/anaisabelvega
mplo2_print.py
Hola
Juan Pedro Maria Luis
Juan | Pedro | Maria | Luis
Juan,Pedro,Maria,Luis.
>>>
```

Entradas de texto por teclado

- El comando input se utiliza para leer información de entrada, desde el teclado.

```
# Comando input

print('Cual es tu nombre? ')
nombre = input()
print('Hola ', nombre, 'Bienvenido al curso !!')


# Otra forma
nombre = input('Cual es tu nombre? ')
print('Hola ', nombre, 'Bienvenido al curso !!')
```

Operadores aritméticos

Operador	Significado
+	Suma
-	Resta
*	Multiplicación
/	División
%	Módulo
**	Potencia
//	División entera

Operadores de asignación

Operador	Significado
+=	Suma
-=	Resta
*=	Multiplicación
/=	División
%=	Módulo
**=	Potencia
//=	División entera

Operadores relacionales

Operador	Significado
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
==	Igual que
!=	Distinto que

Operadores lógicos

- Los operadores lógicos que puedes utilizar son los siguientes:
  - **and**: operador lógico que realiza la operación lógica 'Y' entre dos elementos. El resultado será true si ambos elementos son true, en caso contrario será false.
  - **or**: operador lógico que realiza la operación lógica 'O' entre dos elementos. El resultado será true si uno de los dos elementos es true, en caso contrario será false.
  - **not**: operador lógico que realiza la operación lógica 'NO'. El resultado será true si el elemento es false, y será false si es true.

Tecnofor

Modulo 2:

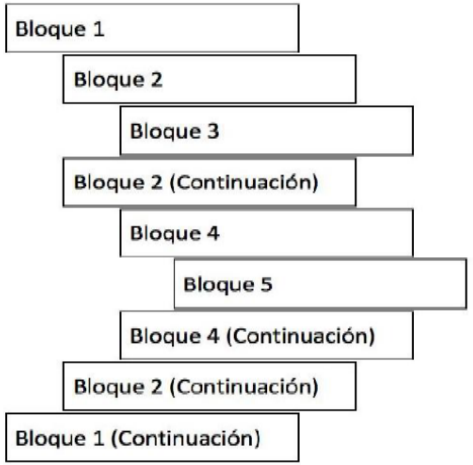
Elementos y operaciones de programación

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL
fundae.es
SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE

Bloques e indentación

- Un bloque es un grupo de sentencias de código fuente que contiene una o más sentencias. Los bloques están delimitados por su inicio y su fin, y la forma de delimitarlos es específica de cada lenguaje de programación.

- Indentación significa mover un bloque de texto hacia la derecha insertando espacios o tabuladores, para así separarlo del margen izquierdo y distinguirlo más fácilmente dentro del texto. Todos los lenguajes utilizan la indentación para aumentar la legibilidad del código fuente, pero en Python no es únicamente una buena práctica estética, ya que utiliza la indentación del código fuente para delimitar los bloques dentro del código fuente, es por eso por lo que es necesario que la utilices de forma correcta.



Condicionales

- **If / elif / else**

```
if numero1>numero2 :
        BloqueInstrucciones1
elif numero1==numero2 :
        BloqueInstrucciones2
else :
        BloqueInstrucciones3
```

Bucles

- **Bucle for**

- En Python, los bucles for se ejecutan sobre elementos iterables, como pueden ser listas, tuplas, cadenas de texto o diccionarios.

for Variable in ColeccionIterable:
BloqueInstrucciones

```
lista = [1,2,3,4,5,6,7,8,9]
for item in lista:
    print(item, end=" ")
```

Bucles

- **Bucle while**

while Condición:
BloqueInstrucciones

```
i = 0
while i<10:
    print(i,end=" ")
    i = i + 1
```

Tecnofor

Modulo 3:

Funciones, tuplas, diccionarios y procesamiento de datos

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL
fundae.es
SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE

Funciones

- Sintaxis:

```
def NombreFuncion (parámetros):
    BloqueInstrucciones
    return ValorRetorno
```

```
def Saludar():
    print("¡Hola Time of Software!")
Saludar()
```

Retornar más de un elemento

- Se pueden devolver más de un elemento con return

```
def SumarRestar(param1, param2):
    return param1 + param2, param1 - param2

numero1 = int(input("Introduce el primer numero: "))
numero2 = int(input("Introduce el segundo numero: "))
resultadosuma, resultadoresta = SumarRestar(numero1,numero2)
print("El resultado de la suma es: ", resultadosuma)
print("El resultado de la resta es: ", resultadoresta)
```

Número variable de argumentos

- En estas funciones los parámetros se reciben como una lista, por lo que tendrás que iterarla para poder procesarlos todos.

```
def Sumar(*valores):
    resultado = 0
    for item in valores:
        resultado = resultado + item
    return resultado

resultado = Sumar(23,56,3,89,78,455)
print("El resultado de la suma es: ", resultado)
```

Listas

- Una lista es un conjunto ordenado de elementos que puede contener datos de cualquier tipo. Es el tipo de colección más flexible de todos.
- Las listas pueden contener elementos del mismo tipo o elementos de diferentes tipos.
- En Python las listas se delimitan con corchetes “[ ]”, con los elementos separados por comas.
- La propiedad len devuelve la longitud, es decir, el numero de elementos que contiene la lista.
- El primer elemento de una lista es el elemento 0, no el 1.

```
lista = ["ordenador","teclado","raton"]
print(len(lista))
print(lista[0])
print(lista[1])
print(lista[2])
```

Listas

- Podemos unir listas a través del operador ‘+’.

```
listaoriginal = ["ordenador","teclado","raton"]
listanueva = ["monitor","impresora","altavoces"]
listafinal = listaoriginal + listanueva
print(listafinal)
```

- Con el operador ‘+’ también podemos añadir un elemento a la lista:

```
lista = ["ordenador","teclado","raton"]
print(lista)
lista = lista + ["mesa"]
print(lista)
```

- Para eliminar un elemento de la lista utilizamos el operador del:

```
lista = ["ordenador","teclado","raton"]
print(lista)
del lista[1]
print(lista)
```

Listas

- Hasta ahora hemos visto listas de una sola dimensión, si queremos listas de dos dimensiones necesitaríamos crear una lista como elemento de otra lista.

```
lista = ["ordenador","teclado","raton", ["tarjeta de sonido","microfono","altavoces"]]
print(lista[0])
print(lista[1])
print(lista[2])
print(lista[3])
print(lista[3][0])
print(lista[3][1])
print(lista[3][2])
```

Tuplas

- Las tuplas son un conjunto de elementos ordenados e inmutables.
- La diferencia con las listas reside en que en las listas puedes manipular los elementos y en las tuplas no. Las tuplas pueden contener elementos del mismo tipo o elementos de diferentes tipos, al igual que las listas.
- En Python las tuplas se delimitan por paréntesis “( )”, con los elementos separados por comas.

```
tupla = ("ordenador","teclado","raton")
print(tupla)
print(len(tupla))
print(tupla[0])
print(tupla[1])
print(tupla[2])
```

Conjuntos

- Un conjunto es una colección de elementos pero que está desordenado es decir no hay un índice

```
# Conjuntos
```

```
conjunto_colores = {"rojo","verde","azul"}
```

```
conjunto_colores
```

```
{'azul', 'rojo', 'verde'}
```

```
for color in conjunto_colores:
    print(color)
```

```
azul
rojo
verde
```

```
conjunto_colores[0]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-12-6ccc7b158046> in <module>
----> 1 conjunto_colores[0]

TypeError: 'set' object does not support indexing
```

```
con
```

Diccionarios

- Los diccionarios son colecciones de elementos compuestos por una clave y un valor asociado. Las claves en los diccionarios no pueden repetirse.
- En Python los diccionarios se delimitan por corchetes “{ }”, con los
- elementos separados por comas y la clave separada del valor mediante dos puntos.

```
mesestraducidos = {"Enero" : "January",
                   "Febrero" : "February",
                   "Marzo" : "March",
                   "Abril" : "April",
                   "Mayo" : "May",
                   "Junio" : "June",
                   "Julio" : "July",
                   "Agosto" : "August",
                   "Septiembre" : "September",
                   "Octubre" : "October",
                   "Noviembre" : "November",
                   "Diciembre" : "December"}
print(mesestraducidos["Noviembre"])
print(mesestraducidos["Mayo"])
```

Tecnofor

Modulo 4:

Módulos, paquetes, métodos y excepciones

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL
fundae.es
SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE

Tipos de excepciones

- En Python existen diferentes tipos de excepciones que pueden controlarse, todas ellas derivan de una serie de excepciones base.
    - Exception: tipo de excepción más genérica, de ella derivan todas las excepciones existentes en Python.
    - ArithmeticError: tipo de excepción genérica para errores aritméticos.
    - BufferError: tipo de excepción genérica para errores relacionados con buffers.
    - LookupError: tipo de excepción genérica para errores relacionados con acceso a datos de colecciones.

try-except

```
try:
    print(3/0)
except:
    print("ERROR: Division por cero")
```

try-except-finally

```
print("¡Iniciando programa!")
try:
    print(3/0)
except:
    print("ERROR: Division erronea")
finally:
    print("¡Programa acabado!")
```

try-except-else-finally

- Consiste en añadir un bloque de instrucciones else que se ejecuta cuando no se lanzan excepciones y no se desea incluir ese bloque de código dentro del bloque final.

```
print("¡Iniciando programa!")
try:
    print(3/1)
except:
    print("ERROR: Division erronea")
else:
    print("¡No se han producido errores!")
finally:
    print("¡Programa acabado!")
```

Captura varias excepciones

```
print("¡Iniciando programa!")
try:
    print(3/0)
except ZeroDivisionError:
    print("ERROR: Division por cero")
except:
    print("ERROR: General")
else:
    print("¡No se han producido errores!")
finally:
    print("¡Programa acabado!")
```

Módulos Python

- Un módulo es un fichero que contiene un conjunto de funciones que puedes incluir y usar en tu aplicación.
- Declaración del módulo

```
modulo1.py
# Modulo

def saludar(nombre):
    print("Hola, soy " + nombre)
```

- Uso del módulo

```
modulo1.py    miprograma.py
1  # mi programa que llamara al modulo modulo1.p
2
3  import modulo1
4
5  modulo1.saludar("Antonio")
6
7  nombre = "Luis"
8  modulo1.saludar(nombre)
9
10
```

Módulos Python

- Otra forma de utilizar los módulos
- Declaración de otra función

modulo1.py

```
# Modulo

def saludar(nombre):
    print("Hola, soy " + nombre)

def despedirse(nombre):
    print("Adios " + nombre)
```

- Uso del módulo

miprograma2.py

```
from modulo1 import despedirse

minombre = "Antonio"
despedirse(minombre)
```

- Con alias

miprograma2.py

```
from modulo1 import despedirse as adios

minombre = "Antonio"

adios(minombre)
```

PIP

- pip es un gestor de paquetes y módulos para Python. Un módulo es una librería de código Python que puedes incluir en tu proyecto.
- Para ver si tenemos pip instalado, desde la terminal:

python3 –m pip –V (mayuscula)

- Para ver la lista de modulos instalados:

python3 -m pip list

- Para instalar un modulo:

python3 -m pip install nombre_modulo

- Podemos desinstalar un modulo:

python3 -m pip uninstall nombre_modulo

Funciones con cadenas de texto

- Para manejar cadenas de texto podemos hacer uso de las siguientes funciones:
  - Función capitalize
  - Función upper
  - Función lower
  - Función len
  - Función isalnum
  - Función isalpha
  - Función isdigit
  - Función islower
  - Función isupper
  - Funciónes lstrip, rstrip, strip
  - Funciones max y min
  - Funcion replace
  - Función swapcase
  - Función split

Función capitalize

- Pone la primera letra, solo la primera, en mayúsculas.

```
cadenaejemplo = "en un lugar de la mancha..."
print(cadenaejemplo.capitalize())
```

Función upper

- Pone en mayúsculas toda la cadena de texto.

```
cadenaejemplo = "en un lugar de la mancha..."
print(cadenaejemplo.upper())
```

Función lower

- Pone en minúsculas toda la cadena de texto.

```
cadenaejemplo = "EN UN LUGAR DE LA MANCHA..."
print(cadenaejemplo.lower())
```

Función len

- Permite saber el número de caracteres que componen la cadena de texto.

```
cadenaejemplo = "En un lugar de la mancha..."
print(len(cadenaejemplo))
```

Función isalnum

- Comprueba si todos los caracteres que componen la cadena de texto son alfanuméricos o no. Devuelve un valor booleano.

```
cadenaejemplo = "En un lugar de la mancha..."
print(cadenaejemplo.isalnum())
cadenaejemplo = "1234567890"
print(cadenaejemplo.isalnum())
cadenaejemplo = "abcdefg1234567890"
print(cadenaejemplo.isalnum())
cadenaejemplo = "abcdefg 1234567890"
print(cadenaejemplo.isalnum())
```

- los caracteres del punto y del espacio en blanco que contienen las cadenas primera y cuarta no son caracteres alfanuméricos.

Función isalpha

- Comprueba si todos los caracteres de la cadena de texto son caracteres alfabéticos.
- Ni los números, ni los puntos ni los espacios en blanco son caracteres alfabéticos, únicamente las letras componen el conjunto de caracteres alfabéticos.

```
cadenaejemplo = "Enunlugardelamancha"
print(cadenaejemplo.isalpha())
cadenaejemplo = "En un lugar de la mancha"
print(cadenaejemplo.isalpha())
cadenaejemplo = "1234567890"
print(cadenaejemplo.isalpha())
cadenaejemplo = "abcdefg 1234567890"
print(cadenaejemplo.isalpha())
```

Función isdigit

- Comprueba si todos los caracteres de la cadena de texto son caracteres numéricos.

```
cadenaejemplo = "En un lugar de la mancha"
print(cadenaejemplo.isdigit())
cadenaejemplo = "1234567890"
print(cadenaejemplo.isdigit())
cadenaejemplo = "abcdefg 1234567890"
```

Función islower

- Comprueba si todos los caracteres que componen la cadena están en minúscula.

```
cadenaejemplo = "En un lugar de la mancha"
print(cadenaejemplo.islower())
cadenaejemplo = "en un lugar de la mancha"
print(cadenaejemplo.islower())
```

Función isupper

- Comprueba si todos los caracteres que componen la cadena de texto están en mayúscula.

```
cadenaejemplo = "En un lugar de la mancha"
print(cadenaejemplo.isupper())
cadenaejemplo = "EN UN LUGAR DE LA MANCHA"
print(cadenaejemplo.isupper())
```

Función lstrip, rstrip, strip

- Para eliminar los caracteres del comienzo de la cadena tienes que utilizar lstrip, para eliminar los caracteres del final de la cadena tienes que utilizar rstrip, y por último, para eliminar ambos a la vez tienes que utilizar strip, que hace lo mismo que los dos anteriores pero en una sola instrucción.

```
cadenaejemplo = " En un lugar de la mancha"
print(cadenaejemplo.lstrip())
cadenaejemplo = "En un lugar de la mancha "
print(cadenaejemplo.rstrip())
cadenaejemplo = " En un lugar de la mancha "
print(cadenaejemplo.strip())
```

Funciones max y min

- permiten conocer el carácter alfabético mayor y menor de la cadena de texto.

```
cadenaejemplo = "abcdefghijklmnopqrstuvwxyz"
print(max(cadenaejemplo))
print(min(cadenaejemplo))
```

Función replace

- Va a permitir reemplazar caracteres de la cadena de texto por otros caracteres.
- En el siguiente ejemplo se reemplaza la A por una E.

```
cadenaejemplo = "AEIOU"
print(cadenaejemplo.replace('A','E'))
```

Función swapcase

- Va a permitir invertir las mayúsculas y minúsculas de la cadena de texto, es decir, las mayúsculas pasarán a ser minúsculas y las minúsculas pasarán a ser mayúsculas.

```
cadenaejemplo = "En un lugar de la mancha"
print(cadenaejemplo.swapcase())
```

Función split

- Va a permitir convertir una cadena de texto en una lista de elementos que se encuentran separados por espacios en la cadena de texto original.

```
cadenaejemplo = "En un lugar de la mancha"
print(cadenaejemplo.split())
```

- Podemos indicar el carácter que tiene que utilizar para separar los elementos de la lista.

```
cadenaejemplo = "31/12/2017"
print(cadenaejemplo.split("/"))
```

Tecnofor

Modulo 5:

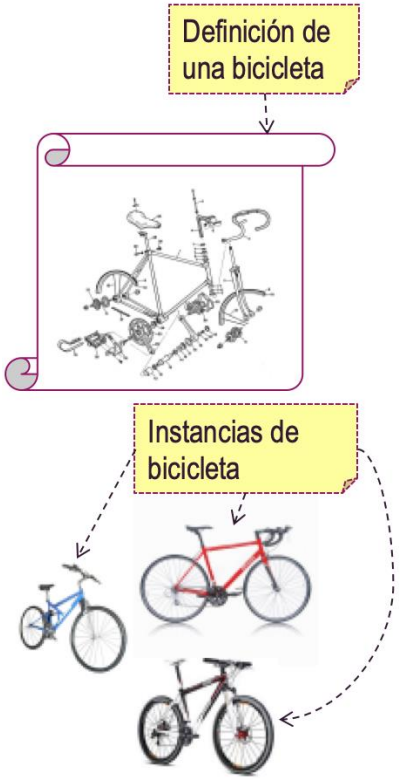
Introducción a la POO: clases, objetos, excepciones y archivos

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL
fundae.es
SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE

Programación OO

- Clase:
  - Una clase es la definición de las características concretas de una determinada tipología de objetos. Es decir, cuáles son los atributos y métodos de los que van a disponer todos los objetos de ese tipo. Equivale a un tipo de dato.

- Objeto:
  - Un objeto es una agregado de atributos (información) y métodos (comportamiento), creado según la definición de una clase.
  - A un objeto concreto que pertenece a una clase se le suele llamar instancia.



Clases

- Clases es uno de los conceptos con más definiciones en la programación, pero en resumen sólo son la representación de un objeto.
- Para definir la clase usas class y el nombre. En caso de tener parámetros los pones entre paréntesis.
- Para crear un constructor haces una función dentro de la clase con el nombre init y de parámetros self (significa su clase misma),

```
class Punto:
    def __init__(self,x,y):
        self.X = x
        self.Y = y
    def MostrarPunto(self):
        print("El punto es (",self.X,",",self.Y,")")

p1 = Punto(4,6)
p1.MostrarPunto()
```

Composición

- Consiste en dividir el software en diferentes componentes, resolver cada uno por separado y por último unirlos en una solución única.
- Ante un problema complejo es más fácil resolverlo cuando se divide en piezas manejables.



Composición

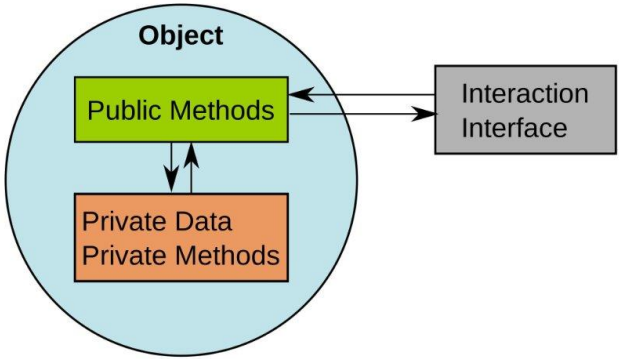
- Composición: Clases como atributos de otras clases

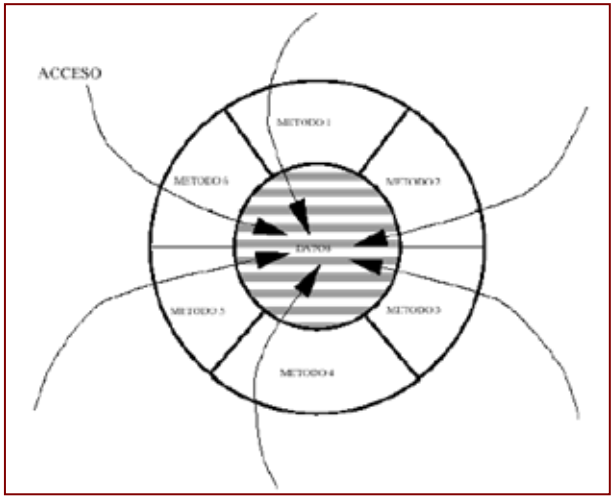
```
class Punto:
    def __init__ (self, x, y):
        self.X = x
        self.Y = y
    def MostrarPunto(self):
        print("El punto es (",self.X,",",self.Y,")")

class Triangulo:
    def __init__ (self, v1,v2,v3):
        self.V1 = v1
        self.V2 = v2
        self.V3 = v3
    def MostrarVertices(self):
        self.V1.MostrarPunto()
        self.V2.MostrarPunto()
        self.V3.MostrarPunto()
```

```
v1 = Punto(3,4)
v2 = Punto(6,8)
v3 = Punto(9,2)
triangulo = Triangulo(v1,v2,v3)
triangulo.MostrarVertices()
```

Encapsulamiento





- Identifica y separa lo que varía o puede variar de lo que permanece inalterado.

Encapsulamiento

- La definición de atributos privados se realiza incluyendo los caracteres “__” entre la palabra “self.” y el nombre del atributo.

```
class PuntoPublico:
    def __init__ (self, x, y):
        self.X = x
        self.Y = y

class PuntoPrivado:
    def __init__ (self, x, y):
        self.__X = x
        self.__Y = y
    def GetX(self):
        return self.__X
    def GetY(self):
        return self.__Y
    def SetX(self, x):
        self.__X = x
    def SetY(self, y):
        self.__Y = y
```

```
publico = PuntoPublico(4,6)
privado = PuntoPrivado(7,3)
print("Valores punto publico:", publico.X,",",publico.Y)
print("Valores punto privado:", privado.GetX(),",",privado.GetY())
publico.X = 2
privado.SetX(9)
print("Valores punto publico:", publico.X,",",publico.Y)
print("Valores punto privado:", privado.GetX(),",",privado.GetY())
```

Encapsulamiento

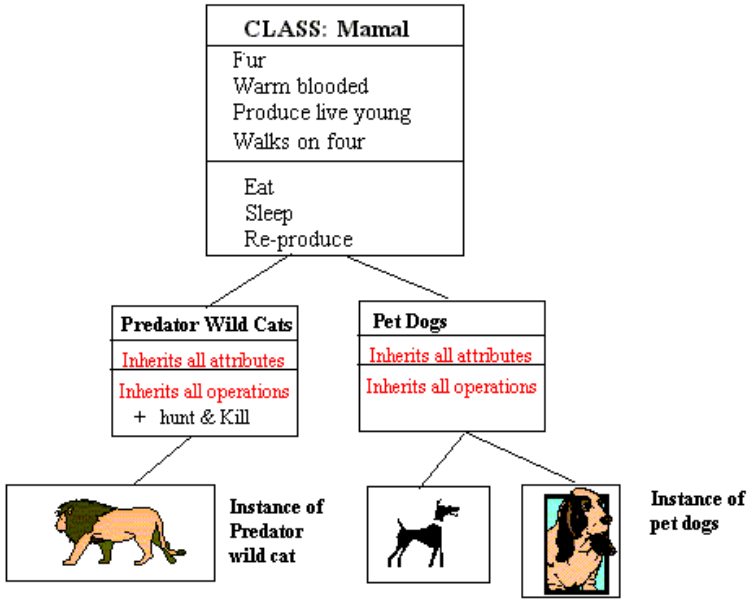
- La definición de métodos privados se realiza incluyendo los caracteres “__” delante del nombre del método.

```
class OperarValores:
    def __init__ (self, v1, v2):
        self.__V1 = v1
        self.__V2 = v2
    def __Sumar(self):
        return self.__V1 + self.__V2
    def __Restar(self):
        return self.__V1 - self.__V2
    def Operar(self):
        print("El resultado de la suma es: ",self.__Sumar())
        print("El resultado de la resta es: ",self.__Restar())

operarvalores = OperarValores(7,3)
operarvalores.Operar()
```

Herencia

- Las clases no están aisladas, sino que se relacionan entre sí, formando una jerarquía de clasificación.
- Los objetos heredan las propiedades, relaciones y los comportamientos de todas las clases a las que pertenecen.
- La herencia organiza y facilita el polimorfismo y el encapsulamiento, permitiendo compartir y extender las propiedades y el comportamiento sin tener que volver a implementarlo.



Herencia

```
class Electrodomestico:
    def __init__ (self):
        self.__Encendido = False
        self.__Tension = 0
    def Encender(self):
        self.__Encendido = True
    def Apagar(self):
        self.__Encendido = False
    def Encendido(self):
        return self.__Encendido
    def SetTension(self, tension):
        self.__Tension = tension
    def GetTension(self):
        return self.__Tension
```

```
class Lavadora(Electrodomestico):
    def __init__ (self):
        self.__RPM = 0
        self.__Kilos = 0
    def SetRPM(self, rpm):
        self.__RPM = rpm
    def SetKilos(self, kilos):
        self.__Kilos = kilos
    def MostrarLavadora(self):
        print("#########")
        print("Lavadora:")
        print("\tRPM:",self.__RPM)
        print("\tKilos:",self.__Kilos)
        print("\tTension:",self.GetTension())
        if self.Encendido():
            print("\t¡Lavadora encendida!")
        else:
            print("\tLavadora no encendida.")
        print("#########")
```

Herencia múltiple



Herencia múltiple

- En Python si que se permite la herencia múltiple, clase que hereda de varias clases.

```
class Negocio(Hotel, Restaurante):
    def __init__(self):
        self.__Nombre = ""
        self.__Direccion = ""
        self.__Telefono = 0
    def SetNombre(self, nombre):
        self.__Nombre = nombre
    def SetDireccion(self, direccion):
        self.__Direccion = direccion
    def SetTelefono(self, telefono):
        self.__Telefono = telefono
    def MostrarNegocio(self):
        print("#########")
        print("Negocio:")
        print("\tNombre:", self.__Nombre)
        print("\tDireccion:", self.__Direccion)
        print("\tTelefono:", self.__Telefono)
        self.MostrarHotel()
        self.MostrarRestaurante()
        print("#########")
```

Ficheros

- La lectura y escritura de ficheros de texto en Python se realiza con la función open, que devuelve un objeto que te permitirá realizar dichas operaciones con el fichero que has abierto. Es realmente sencillo trabajar con ficheros de texto en Python.
- La función open tiene dos parámetros de entrada:
    - Ruta del fichero que se desea abrir.
    - Modo de apertura del fichero.

Ficheros

- Modos de apertura disponibles:
  - “r”: abre el fichero para lectura. Es el modo de apertura por defecto en el caso de que no se especifique uno.
  - “w”: abre el fichero para escritura truncándolo, es decir, borrando todo el contenido que tiene para empezar a escribir de nuevo desde cero.
  - “x”: crea un fichero para escribir en él. En caso de que ya exista devuelve un error.
  - “a”: abre el fichero para escritura situando el cursor de escritura al final del fichero.
  - “b”: abre el fichero en modo binario. Un fichero binario es un tipo de fichero con información representada en ceros y unos en lugar de texto plano, por ejemplo, fotografías, archivos ejecutables, ficheros de Microsoft Word, etc.
  - “t”: abre el fichero en modo fichero de texto. Es el modo de apertura por defecto en el caso de que no se especifique que sea binario o de texto.
  - “+”: abre el fichero para lectura y escritura.

Ficheros de texto

- Leer fichero

```
fichero = open("ficherotexto.txt","rt")
datos_fichero = fichero.read()
print(datos_fichero)
```

- Grabar fichero

```
fichero = open("fichero_para_grabar.txt","wt")
texto_del_fichero = "Hola, esta es la línea que vamos a grabar en el fichero de texto"
fichero.write(texto_del_fichero)
fichero.close()
```

Ficheros de texto

- Añadir datos a un fichero

```
fichero = open("ficherotexto.txt","at")
cadena_para_incluir = "\nEsta es la tercera fila del fichero"
fichero.write(cadena_para_incluir)
fichero.close()
```

- Borrar fichero

```
from os import remove
from os import path

if path.exists("Proyecto8_FicherosTexto/fichero_escritura.txt"):
    remove("Proyecto8_FicherosTexto/fichero_escritura.txt")
```

Tecnofor

Muchas gracias por vuestra asistencia

C/ Manuel Tovar, 31 28034-Madrid

GOBIERNO DE ESPAÑA
MINISTERIO DE TRABAJO, MIGRACIONES Y SEGURIDAD SOCIAL

fundae.es

SERVICIO PÚBLICO DE EMPLEO ESTATAL
SEPE